AULA PRÁTICA · GIT E GITHUB

Versionamento de código na prática

Comandos essenciais e boas práticas para trabalhar em equipe com Git e GitHub

```
$ git switch -c fix/menor-nota
$ git commit -m "fix: inicializa menor com a primeira nota"
$ git push -u origin fix/menor-nota
```

ROTEIRO

O que vamos fazer hoje

01

Entender o porquê

O que é versionamento e qual é a diferença entre Git e GitHub.

02

Dominar o ciclo básico

status, add, commit, push e pull: os comandos do dia a dia.

03

Colaborar como um time

Branches, issues, pull requests, revisão de código e conflitos.

04

Corrigir 14 bugs

Atividade com código Python e testes automáticos, usando o fluxo profissional.

CONCEITOS

Por que versionar código

Situação	Sem versionamento	Com Git e GitHub
Versões	trabalho_final_v2_AGORA_vai.py	Um arquivo, todo o histórico de versões
Erros	“Quebrei tudo, como eu voltava?”	Voltar a qualquer commit em segundos
Autoria	“Quem mexeu nessa linha?”	Cada linha tem autor, data e motivo
Equipe	Código enviado por e-mail ou pen drive	Branches, pull requests e revisão

CONCEITOS

Git e GitHub: qual é a diferença

Git

Ferramenta de controle de versão que roda no seu computador.

- Guarda o histórico de commits
- Cria branches e faz merges
- Funciona sem internet

GitHub

Plataforma que hospeda repositórios Git na nuvem.

- Repositórios remotos e backup
- Issues, pull requests e revisão
- Automação e colaboração em equipe

O Git é a ferramenta; o GitHub é o lugar onde o time se encontra.

CONCEITO-CHAVE

O caminho de uma alteração



Voltando atrás: **git restore** descarta alterações, **git restore --staged** tira do staging e **git pull** traz as novidades do GitHub.

PRIMEIROS PASSOS

Configuração inicial

```
# uma única vez em cada computador
$ git config --global user.name "Seu Nome"
$ git config --global user.email "email-da-conta-github@exemplo.com"
$ git config --global init.defaultBranch main
$ git config --list
```



Autenticação: o GitHub não aceita mais senha no git push. Use **gh auth login**, um token de acesso pessoal (PAT) ou uma chave SSH.

PRIMEIROS PASSOS

Trazendo o projeto para o seu computador

```
# 1. No GitHub, clique em Fork (cria uma cópia na sua conta)
$ git clone https://github.com/<seu-usuario>/sudoku-game.git
$ cd sudoku-game
$ git remote add upstream https://github.com/josepaes23/sudoku-game.git
$ git remote -v
```

origin

O seu fork no GitHub, onde você tem permissão de escrita.

upstream

O repositório original da aula, de onde você busca as novidades.

Se o professor adicionou você como colaborador, pule o fork: clone direto e use apenas o origin.

CICLO BÁSICO

Os comandos do dia a dia

`git status`	O que mudou e o que está preparado
`git diff`	Suas alterações ainda não preparadas
`git add gerenciador_notas.py`	Prepara o arquivo para o commit
`git diff --staged`	Revê o que entrará no commit
`git commit -m "mensagem"`	Registra o commit no repositório local
`git push`	Envia seus commits para o GitHub
`git pull`	Traz as novidades do GitHub

BOAS PRÁTICAS

Mensagens de commit que contam uma história

```
tipo: descrição curta no imperativo
```

`feat` `fix` `docs` `test` `refactor` `chore`

⚠ Evite

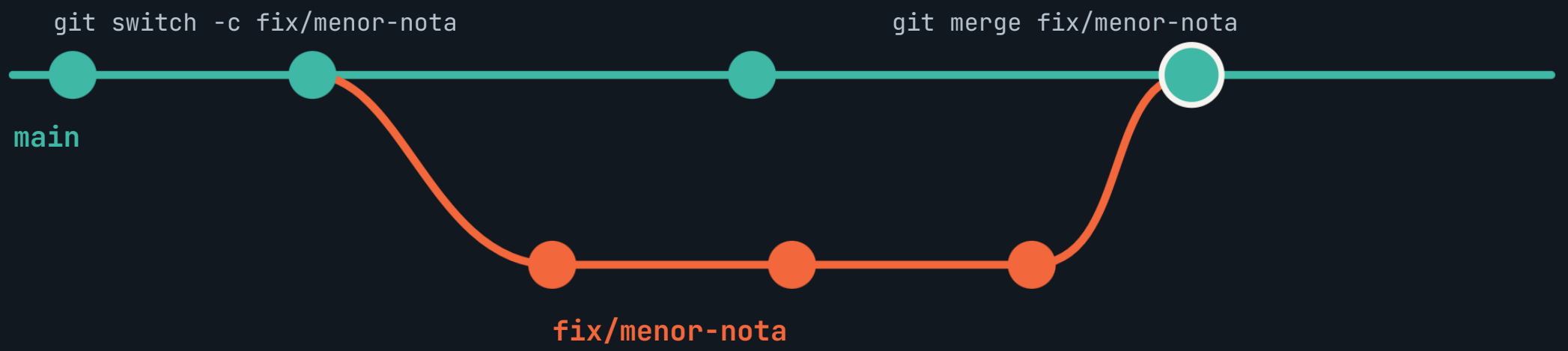
```
ajustes
agora vai
teste 2
asdf
```

✓ Prefira

```
fix: usa divisão real em percentual_faltas
feat: adiciona média ponderada
docs: explica como rodar os testes
```

BRANCHES

Trabalhando em paralelo sem se atrapalhar



main
Sempre estável: é o código que o time pode usar.

branch
Espaço seguro para corrigir sem afetar os outros.

merge
Junta o trabalho na main depois da revisão.

BRANCHES

Comandos de branch

`git branch`	Lista as branches (a atual vem com *)
`git switch -c fix/menor-nota`	Cria a branch e já troca para ela
`git switch main`	Volta para a main
`git push -u origin fix/menor-nota`	Envia a branch para o GitHub
`git merge fix/menor-nota`	Junta a branch na branch atual
`git branch -d fix/menor-nota`	Apaga a branch já integrada

FLUXO DA AULA

Da issue ao merge

1

Issue

Registre o bug e o teste que falha.

```
gh issue create
```

2

Branch

Uma branch por issue, a partir da main atualizada.

```
git switch -c fix/menor-nota
```

3

Commit

Corrija, rode os testes e registre uma mudança.

```
git commit -m "fix: …"
```

4

Push

Envie a branch para o seu fork no GitHub.

```
git push -u origin fix/…
```

5

Pull Request

Peça revisão e escreva Closes #número.

```
gh pr create
```

6

Merge

Após a aprovação, integre e apague a branch.

```
git branch -d fix/…
```

COLABORAÇÃO

Fork, origin e upstream

upstream
`josepaes23/sudoku-game`
O repositório original da aula

Fork →

origin
`seu-usuario/sudoku-game`
O seu fork, no GitHub

git clone →

local
`~/sudoku-game`
A cópia no seu computador

```
# atualizar a sua main com a do repositório original
$ git pull upstream main
$ git push origin main
```

COLABORAÇÃO

Pull request: propor, revisar, integrar

Um bom PR tem

- Título claro, no padrão dos commits
- Descrição: o que mudou, por quê e como testar
- Closes #3 para fechar a issue sozinho
- Testes passando
- Poucas linhas alteradas

```
$ git push -u origin fix/menor-nota
$ gh pr create \
  --title "fix: corrige menor_nota" \
  --body "Closes #3"
$ gh pr list
$ gh pr merge
```

COLABORAÇÃO

Sincronizando com o GitHub

COMANDO	O QUE FAZ	QUANDO USAR
`git fetch`	Baixa as novidades, sem alterar seus arquivos	Quando quer olhar antes de integrar
`git pull`	fetch + merge na branch atual	Uso diário, o mais comum
`git pull --rebase`	fetch + reaplica seus commits por cima	Histórico linear, sem commit de merge

Regra de ouro: faça git pull antes de começar a trabalhar e antes de dar git push.

CONFLITOS

Quando duas branches mexem nas mesmas linhas

```
<<<<<<< HEAD
    if media_a < media_b:
=======
    if media_b > media_a:
>>>>>>> fix/ranking-ordem
```

`<<<<<<< HEAD`	A versão da sua branch atual
`=======`	Separa as duas versões
`>>>>>>> nome`	A versão que vem da outra branch

```
CONFLICT (content): Merge conflict in gerenciador_notas.py
```

CONFLITOS

Resolvendo em cinco passos

1	`git status`	Veja quais arquivos estão em conflito
2	`editar o arquivo`	Escolha o código certo e apague as marcas
3	`python -m unittest`	Confirme que os testes passam
4	`git add gerenciador_notas.py`	Marque o conflito como resolvido
5	`git commit`	Conclui o merge (a mensagem já vem pronta)

⚠ Desistiu no meio? **git merge --abort** volta ao estado anterior ao merge.

DESFAZENDO ERROS

A ferramenta certa para cada situação

`git restore arquivo`	Descarta alterações ainda não preparadas
`git restore --staged arquivo`	Tira do staging, mantendo a alteração
`git commit --amend`	Corrige o último commit (antes do push)
`git reset --soft HEAD~1`	Desfaz o último commit, mantendo tudo preparado
`git revert abc1234`	Cria um commit que desfaz outro (seguro após o push)
`git stash / git stash pop`	Guarda e recupera alterações temporariamente
`git reset --hard`	**Irreversível: descarta tudo. Cuidado!**

INVESTIGANDO

Lendo o histórico do projeto

`git log --oneline --graph --decorate --all`	Histórico em grafo, com todas as branches
`git show abc1234`	O que um commit alterou
`git log -p gerenciador_notas.py`	Histórico de um arquivo, com as mudanças
`git blame gerenciador_notas.py`	Quem alterou cada linha, e quando
`git diff main..fix/menor-nota`	Diferença entre duas branches
`git reflog`	A rede de segurança: recupera commits perdidos

BOAS PRÁTICAS

Commits que facilitam a vida do time

Atômicos

Um commit, uma mudança lógica. Um bug corrigido, um commit.

Funcionando

Nunca commite código que quebra o projeto ou os testes.

Revisados

Use git status e git diff --staged antes de cada commit.

Frequentes

Pequenos e seguidos facilitam revisão, revert e blame.

Cuidado: evite git add . no automático. Você pode levar junto senhas, caches e arquivos que não deveriam ir.

BOAS PRÁTICAS

Branches e pull requests saudáveis

Branches

- A main fica sempre funcionando
- Uma branch por tarefa ou issue
- Nomes com prefixo: fix/, feature/, docs/
- Apague a branch depois do merge

Pull requests

- Pequenos e focados em uma mudança
- Descrição clara, com Closes #número
- Testes passando antes de abrir
- Revise o código, não a pessoa

Em projetos de equipe, ative a **proteção da main** (Settings, Branches): merge só por pull request com revisão.

.GITIGNORE E SEGREDOS

O que nunca deve ir para o repositório

.gitignore (Python)

```
__pycache__/
*.pyc
.venv/
.env
.idea/
.vscode/
```

Segredos ficam de fora

Senhas, tokens e chaves de API. Se vazou, revogue a chave na hora: apagar o commit depois não basta.

Commitou o arquivo errado?

```
git rm --cached arquivo
```

Depois adicione o nome ao .gitignore e faça um novo commit.

PARA LEMBRAR

Commit pequeno, mensagem clara, main sempre funcionando.

MÃO NA MASSA

Atividade: caçando bugs no gerenciador de notas

Um boletim em Python com 14 bugs plantados de propósito. Os testes automáticos dizem se você acertou.

`gerenciador_notas.py`

O código com bugs: variáveis, operadores, condicionais, laços e listas.

`test_gerenciador_notas.py`

31 testes. No início 18 falham; a meta é deixar todos passando.

`README.md`

Guia com o passo a passo, os comandos e as boas práticas.

```
$ python -m unittest -v test_gerenciador_notas
Ran 31 tests in 0.007s
FAILED (failures=16, errors=2)
```

MÃO NA MASSA

Roteiro: um bug, nove passos

1. Rodar os testes	`python -m unittest -v test_gerenciador_notas`
2. Criar a issue	`gh issue create --title "menor_nota retorna 0"`
3. Criar a branch	`git switch -c fix/menor-nota`
4. Corrigir e testar	`python -m unittest test_gerenciador_notas`
5. Commit	`git commit -am "fix: corrige menor_nota"`
6. Push	`git push -u origin fix/menor-nota`
7. Pull request	`gh pr create --fill`
8. Atualizar a main	`git switch main && git pull upstream main`
9. Limpar	`git branch -d fix/menor-nota`

Repita para cada bug, sempre partindo de uma main atualizada. No passo 7, escreva Closes #número na descrição.

MÃO NA MASSA

Desafios extras

Conflito

Duas duplas corrigem o mesmo bug de jeitos diferentes e resolvem no merge.

Desfazer

Faça um commit ruim e desfaça com git revert. Corrija uma mensagem com --amend.

Stash

Troque de branch no meio de uma alteração com git stash e volte com git stash pop.

Investigar

Use git blame e git log -p para descobrir quem mexeu em cada função.

Release

Com todos os testes passando, crie a tag v1.0 e envie ao GitHub.

Proteger a main

Em Settings, Branches, exija pull request e revisão antes do merge.

CHEAT SHEET

Os comandos que você mais vai usar

Ciclo

```
git status
git add arquivo
git commit -m "..."
git push
```

Branches

```
git switch -c nome
git switch nome
git merge nome
git branch -d nome
```

Sincronizar

```
git fetch
git pull
git pull --rebase
git remote -v
```

Desfazer e investigar

```
git restore arquivo
git revert hash
git stash
git log --oneline --graph
```

PARA CONTINUAR

Material de apoio

Repositório da aula	github.com/josepaes23/sudoku-game
Livro Pro Git, em português	git-scm.com/book/pt-br/v2
Documentação do GitHub	docs.github.com/pt
Cursos práticos interativos	skills.github.com
Branches de forma visual	learngitbranching.js.org
Padrão de mensagens de commit	conventionalcommits.org/pt-br

Agora é com vocês: abram o terminal e mãos à obra.